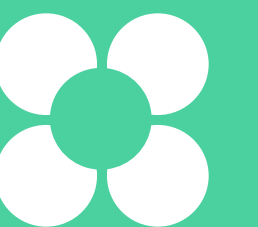


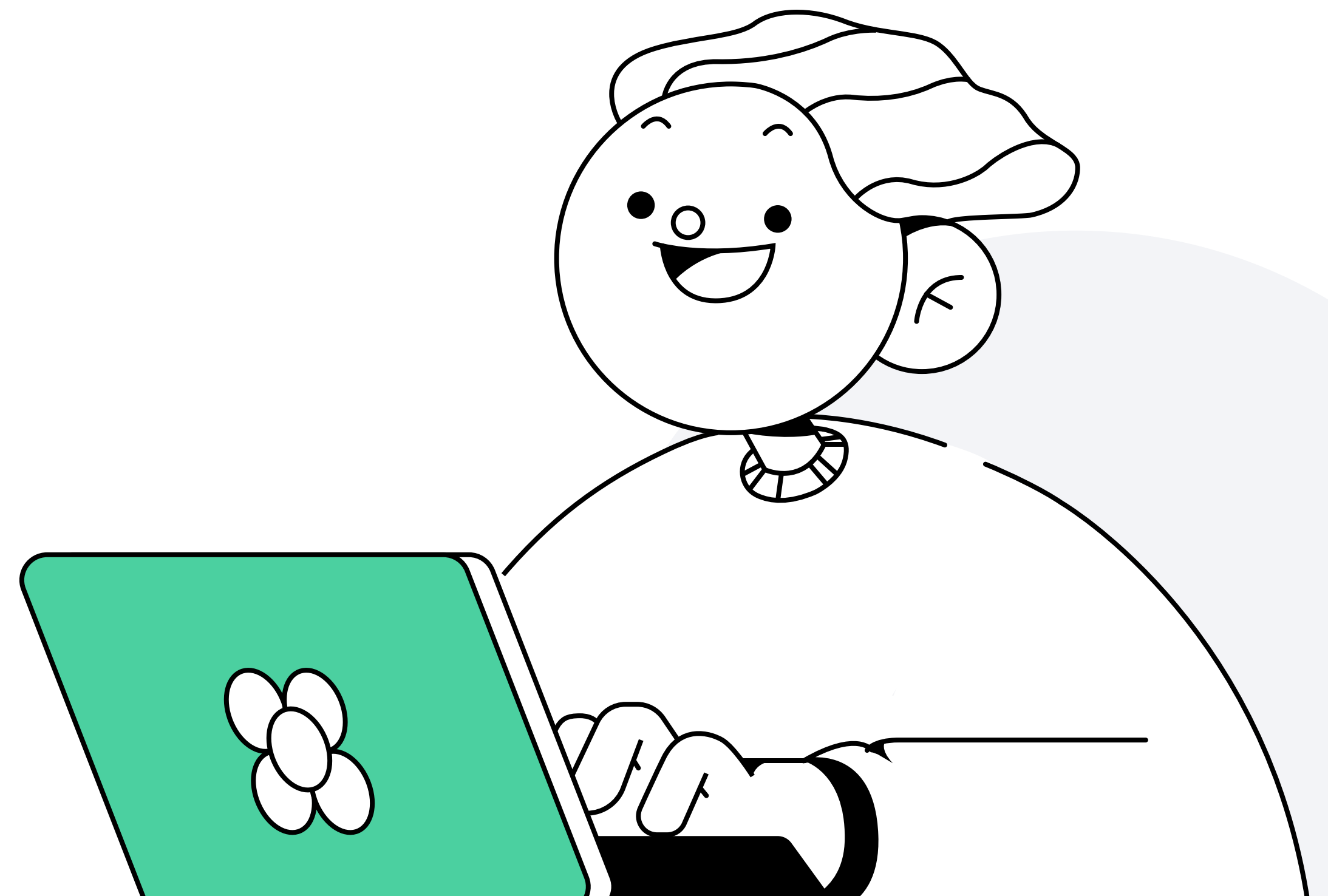
Stream API. ПотокИ, повторные вызовы, основные методы

Денис Коротков
Software Engineer, Harman Connected Service



Цели занятия

- 1 Рассмотреть Stream API, его назначение и использование
- 2 Познакомиться с промежуточными операциями Stream API
- 3 Научиться использовать терминальные операции Stream API



Stream API



Stream API

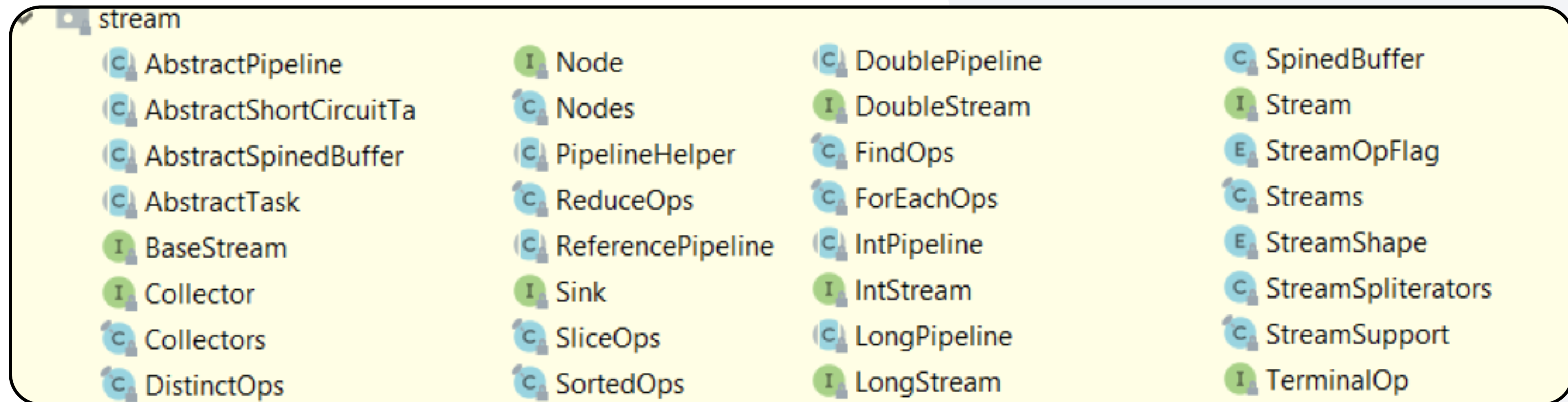
Начиная с JDK 8 в Java появился новый API* — **Stream API**.

Его задача — упростить работу с наборами данных, в частности, упростить операции фильтрации, сортировки, модификации и другие манипуляции с данными

*API (Application Programming Interface) — описание способов, набор классов, процедур, функций, структур или констант, которыми одна компьютерная программа может взаимодействовать с другой программой

Stream API

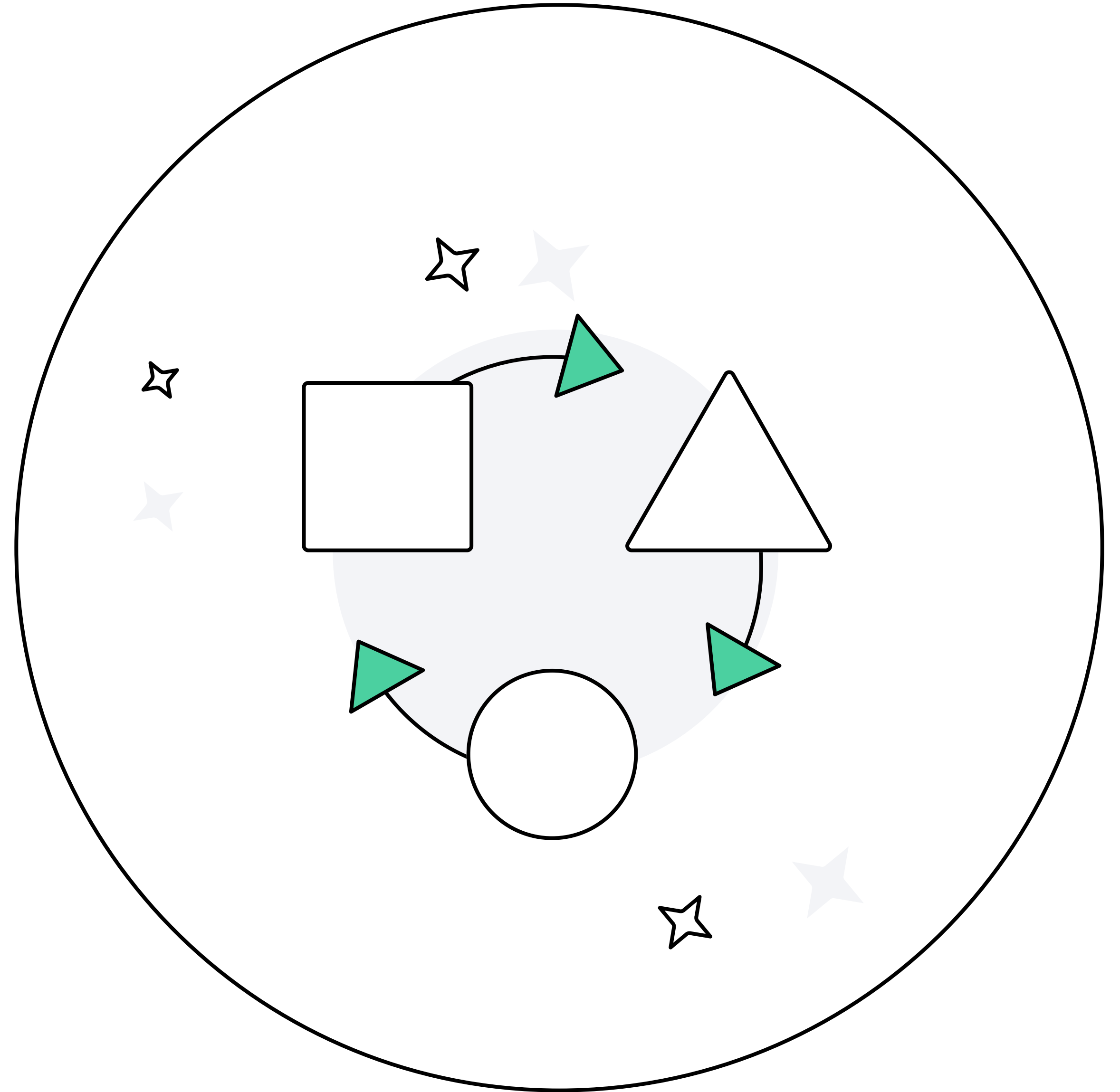
Основная функциональность Stream API сосредоточена в пакете `java.util.stream`



Stream API. Стрим

Канал передачи данных
из источника данных.

В качестве источника могут
выступать файлы, массивы
и коллекции

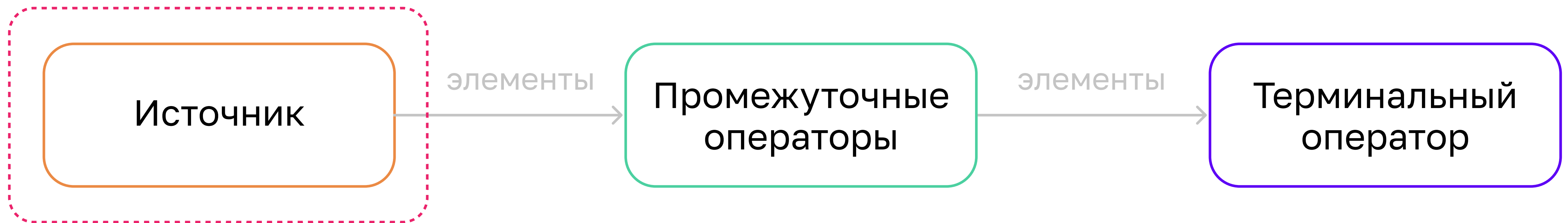


Жизненный цикл стрима

Три стадии:

- создание стрима
- применение к стриму промежуточных операций
- применение к стриму завершающей операции для получения результата

Создание стрима



Способы создания стрима

- Пустой стрим: `Stream.empty()`
- Стрим из List: `list.stream()`
- Стрим из Map: `map.entrySet().stream()`
- Стрим из массива: `Arrays.stream(array)`
- Стрим из указанных элементов: `Stream.of("1", "2", "3")`

Создание стрима

Пример: создать стрима из листа, состоящего из трёх элементов

```
List<String> list = new ArrayList<>();  
list.add("One");  
list.add("Two");  
list.add("Three");  
Stream stream = list.stream();
```

Типы стримов

Кроме объектных стримов `Stream<T>`, существуют специальные стримы для примитивных типов:

- `IntStream` — для `int`
- `LongStream` — для `long`
- `DoubleStream` — для `double`

Для `boolean`, `byte`, `short` и `char` специальных стримов нет. Вместо них можно использовать `IntStream`, а затем приводить к нужному типу.

Для `float` тоже придётся воспользоваться `DoubleStream`.

Примитивные стримы полезны, так как не нужно тратить время на боксинг/анбоксинг, к тому же у них есть специальные операторы, упрощающие жизнь

Операции со стримами

- **Промежуточные** — операции, возвращающие трансформированный стрим. К возвращённому стриму также можно применить промежуточные операции.

Примеры: фильтрация, преобразование, сортировка, удаление дубликатов, ограничение и многое другое.

- **Терминальные** — терминальные операции возвращают конкретный результат, они завершающие.

К терминальным операциям относят: перебор элементов, поиск, нахождение минимума и максимума, расчёт количества и т. д.

Операции со стримами

Задача: найти в массиве количество всех чисел больше 0.

```
long count = IntStream.of(-5, -4, -3, -2, -1, 0, 1, 2, 3, 4, 5)
    .filter(w -> w > 0)
    .count();
System.out.println(count);
```

- 1 Промежуточный метод **filter** принимает стрим чисел и возвращает преобразованный стрим чисел больше 0
- 2 Метод **count** представляет терминальную операцию и возвращает количество чисел в стриме

Стримы vs коллекции

- **Стримы не хранят элементы.** Элементы, используемые в стримах, могут храниться в коллекции либо могут быть напрямую сгенерированы
- **Операции со стримами не изменяют источника данных,** а лишь возвращают новый стрим с результатами этих операций
- **Для стримов характерно отложенное выполнение.** Выполнение всех операций со стримом происходит лишь тогда, когда выполняется терминальная операция и возвращается конкретный результат, а не новый стрим

И снова лямбды

Одна из отличительных черт Stream API – применение **лямбда-выражений**, которые позволяют значительно сократить запись выполняемых действий

И снова лямбды

Задача: отфильтровать из массива значения больше 90, полученные значения увеличить на 10. Вывести на экран результирующий лист.

Пример решения до Java 8:

```
int[] input = {50, 60, 70, 80, 90, 100};
List<Integer> result = new ArrayList<>();
int count = 0;
for (int x : input) {
    if (x >= 90) continue;
    x += 10;
    count++;
    if (count > 3) break;
    result.add(x);
}
System.out.println(result);
```


Стрим для работы над листом интов

```
int[] input = {50, 60, 70, 80, 90, 100};  
List<Integer> result =  
Arrays.stream(input)  
    .filter(x -> x < 90)  
    .map(x -> x + 10)  
    .limit(3)  
    .boxed()  
    .collect(Collectors.toList());  
System.out.println(result);
```

Ещё больше примеров

- 1 Создаём коллекцию list и заполняем её тестовыми данными
- 2 Создаём объект stream, используя list в качестве источника
- 3 Метод filter — промежуточный оператор, элемент коллекции x в котором фильтруется согласно правилу, указанному после символа ->
- 4 Метод map — промежуточный оператор, работающий по аналогии с filter, но необходимый для преобразования данных в коллекции
- 5 Метод forEach — терминальный оператор перебора элементов x, вызывающий System.out.println(x)

```
List<String> list =  
    Arrays.asList("My", "Pen", "Is", "Black");  
list.stream()  
    .filter(x -> x.length() == 3)  
    .map(String::toUpperCase)  
    .forEach(System.out::println); // PEN
```

The diagram illustrates the execution flow of the provided Java code. Numbered callouts are as follows:

- 1: Points to the variable `list` in the first line.
- 2: Points to the `list.stream()` method call.
- 3: Points to the lambda expression `x -> x.length() == 3` in the `filter` method.
- 4: Points to the `map` method call.
- 5: Points to the `forEach` method call.

Что под капотом

- 1 Метод `stream()` создаёт экземпляр класса `Stream`
- 2 Каждая промежуточная операция работает с экземпляром стрима и при этом создаёт новый экземпляр стрима на выходе
- 3 Нельзя дважды использовать один и тот же экземпляр стрима. Он одноразовый
- 4 Терминальная операция не даёт на выходе стрима, но даёт какой-либо результат

```
List<String> list =  
    Arrays.asList("My", "Pen", "Is", "Black");  
Stream<String> stream_1 = list.stream();  
Stream<String> stream_2 = stream_1.filter(x -> x.length() == 3);  
Stream<String> stream_3 = stream_2.map(String::toUpperCase);  
stream_3.forEach(System.out::println); // PEN
```


Имейте в виду

- Обработка стримом коллекции не начнётся до тех пор, пока не будет вызван терминальный оператор

```
list.stream().filter(s -> s > 5);
```

- Экземпляр стрима нельзя использовать более одного раза

```
Stream<String> stream = list.stream();  
stream.filter(x-> x.toString().length() == 3).forEach(System.out::println);  
stream.forEach(x -> System.out.println(x));
```

Exception in thread "main" java.lang.IllegalStateException: stream has already been operated upon or closed

- Промежуточных операторов, вызванных на одном стриме, может быть множество, а терминальный оператор — только один

```
stream.filter(x -> x.toString().length() == 3).  
    map(x -> x + " - the length of the letters is three").  
    forEach(x -> System.out.println(x));
```

Промежуточные операции

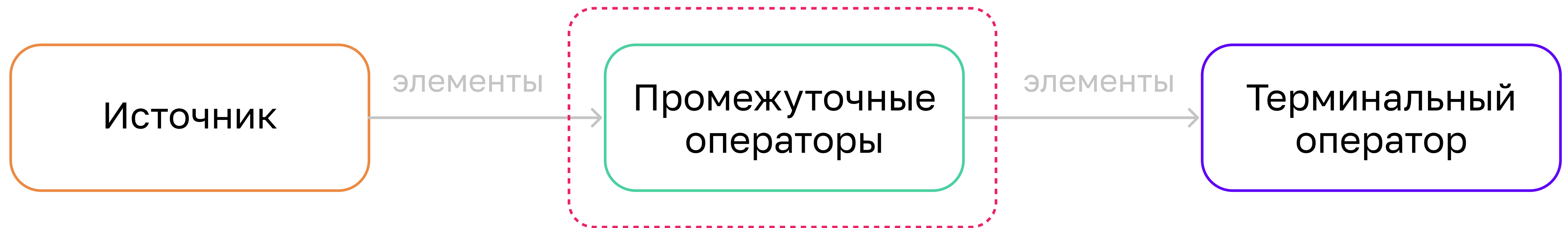


Промежуточные операции

Промежуточные (нетерминальные) **операции** Stream API – операции, которые **преобразовывают** или **фильтруют** элементы в потоке.

Добавляя нетерминальную операцию в стрим, получим новый стрим в качестве результата. Новый стрим будет представлять собой поток элементов, полученных из исходного стрима с применением нетерминальной операции

Промежуточные операции



Filter

Метод фильтра принимает **Predicate**, который вызывается для каждого элемента в стриме:

- Если элемент **должен быть включён** в результирующий стрим, Predicate должен вернуть значение **true**
- Если элемент **не должен быть включён**, Predicate должен вернуть **false**

Filter

Пример: отфильтровать лист. Вывести в консоль город с названием длиной более 3 символов и с наличием буквы «М»

```
List<String> list = Arrays.asList("Moscow", "NY", "Tokyo");
```

```
list.stream()
```

```
    .filter(value -> value.length() >= 3)
```

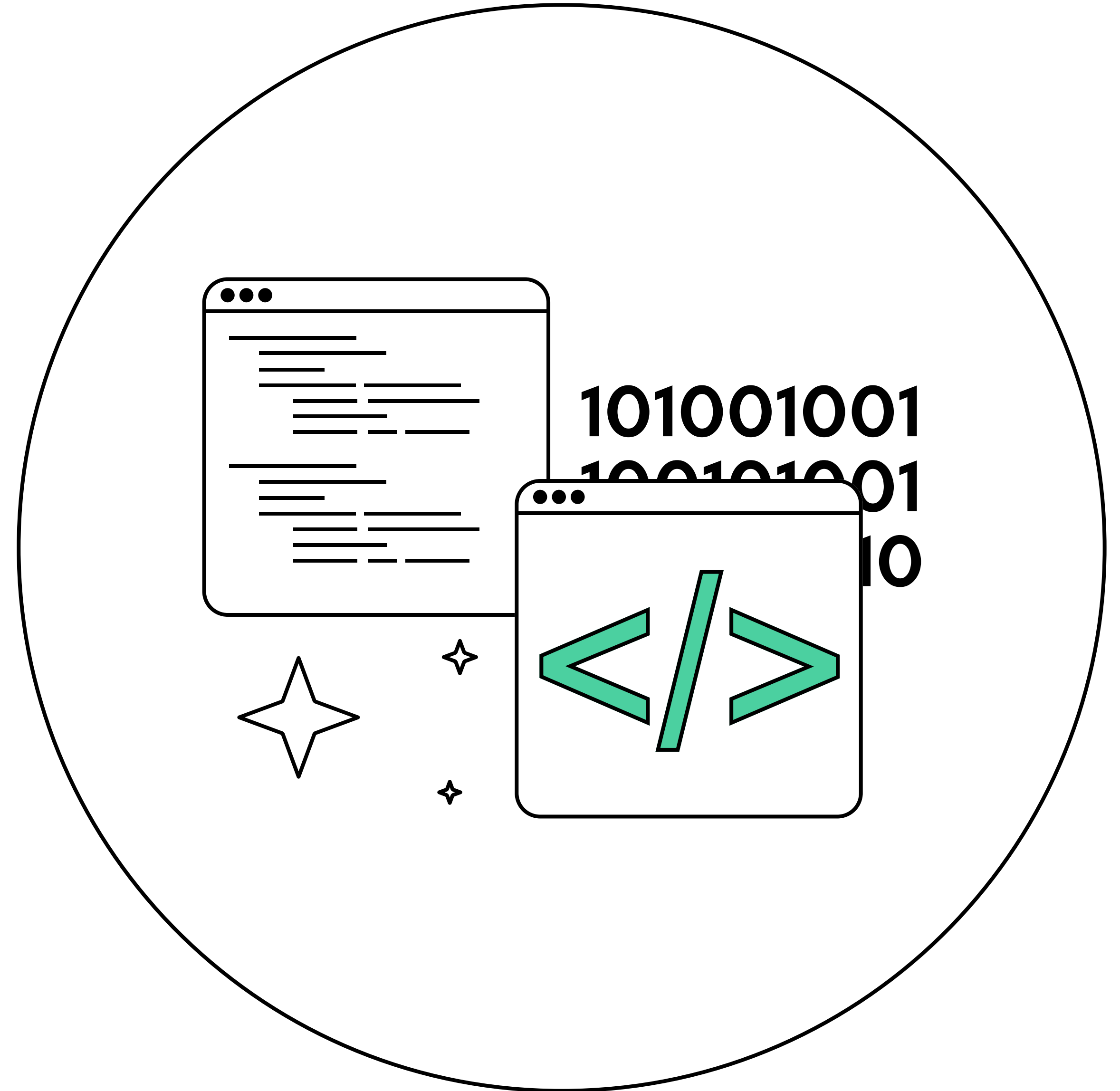
```
    .filter(value -> value.contains("M"))
```

```
    .forEach(System.out::println); // Moscow
```

Map

Маппинг **map** задаёт функцию преобразования одного объекта в другой.

Например, в списке строк можно преобразовать элементы в нижний регистр, верхний регистр или в подстроку исходной строки и т. д.



Map

Пример: преобразуем список городов в слова заглавными буквами и добавим строку

```
List<String> list = Arrays.asList("Moscow", "NY", "Tokyo");
```

```
list.stream()
```

```
    .map(String::toUpperCase)
```

```
    .map(value -> value + " <3")
```

```
    .forEach(System.out::println);
```

```
// MOSCOW <3
```

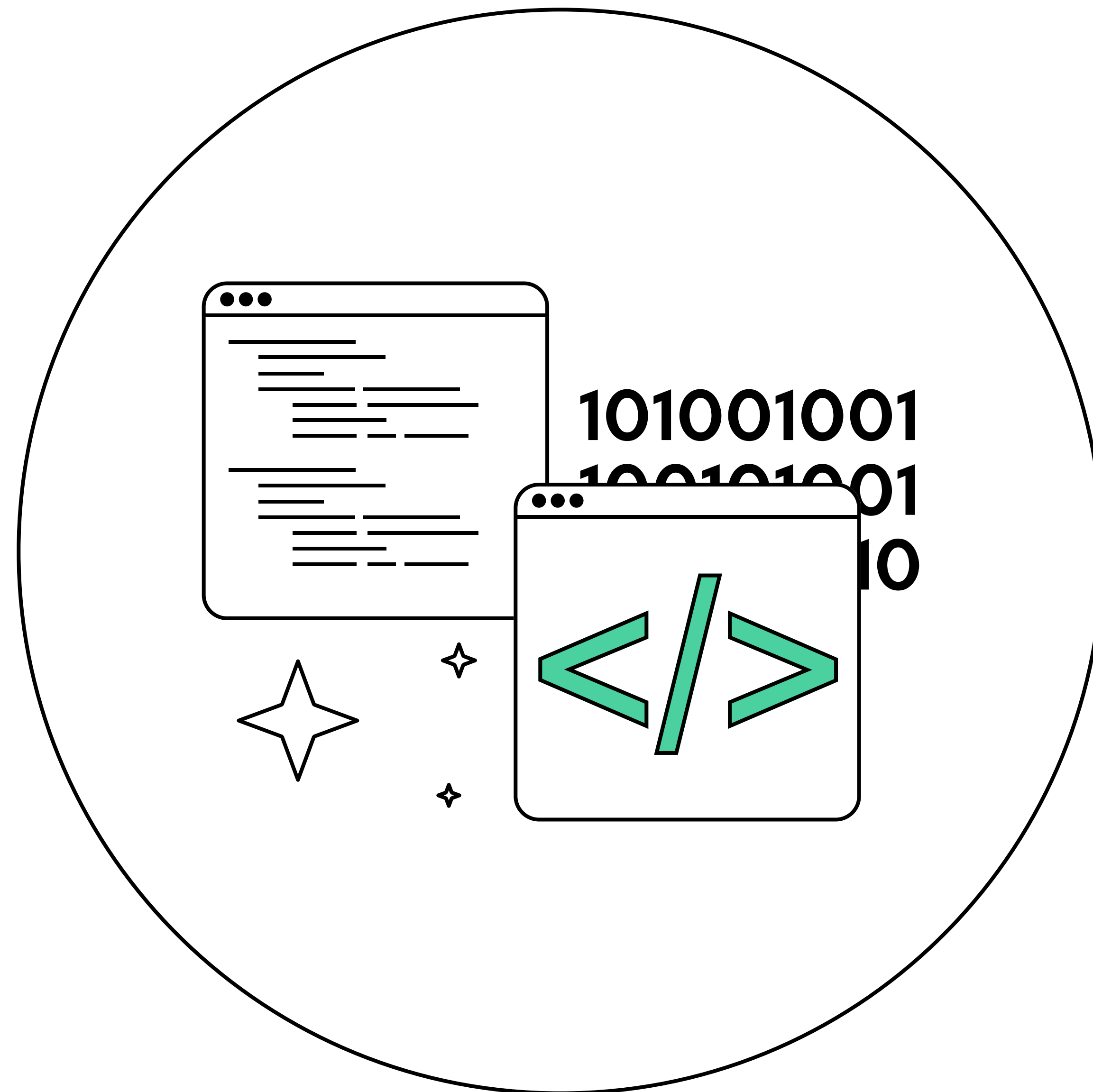
```
// NY <3
```

```
// TOKYO <3
```


FlatMap

Метод **flatMap** отображает один элемент в виде нескольких элементов.

Идея состоит в том, что flatMap сплюсчивает каждый элемент из сложной структуры, состоящей из нескольких внутренних элементов, в плоский стрим, состоящий только из ЭТИХ внутренних элементов



FlatMap

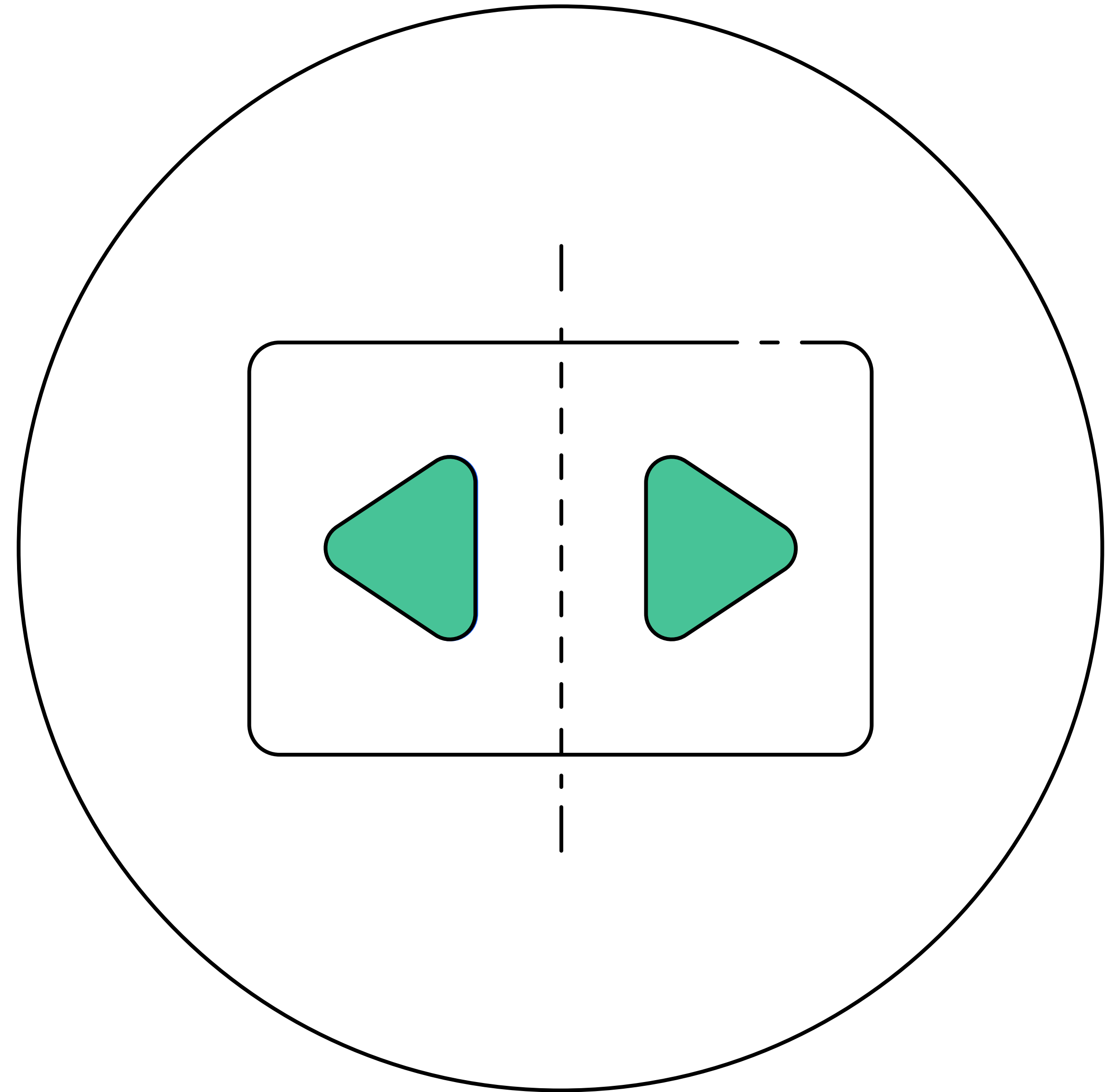
Пример: требуется вывести в консоль количество слов в четверостишии

```
List<String> list = new ArrayList<>();  
list.add("И долго буду тем любезен я народу");  
list.add("Что чувства добрые я лирой пробуждал");  
list.add("Что в мой жестокий век восславил я Свободу");  
list.add("И милость к падшим призывал");  
  
long count = list.stream()  
    .flatMap(value -> Arrays.stream(value.split(" ")))  
    .count();  
System.out.println(count); // 26
```

Distinct

Метод **distinct** применяется для удаления дубликатов.

Метод **distinct** является нетерминальной операцией, которая возвращает новый стрим, который будет содержать только уникальные элементы из исходного стрима. Любые дубликаты будут исключены



Distinct

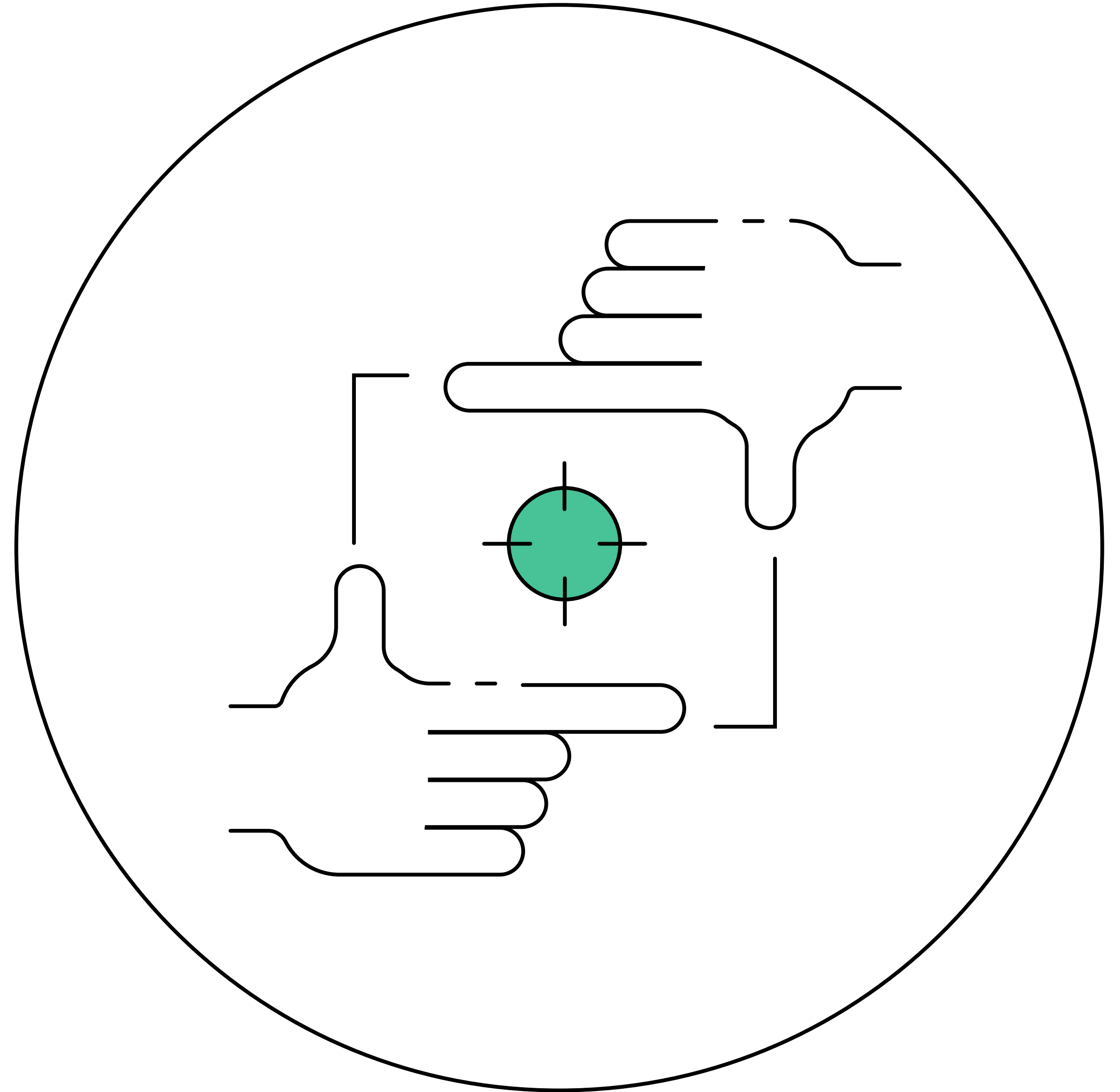
Пример: требуется удалить дубликаты из листа

```
List<String> list = Arrays.asList("one", "two", "three", "one", "two");  
list = list.stream()  
    .distinct()  
    .collect(Collectors.toList());  
System.out.println(list); // [one, two, three]
```

Limit

Метод **limit** применяется для ограничения количества элементов в стриме.

Метод **limit** возвращает новый стрим, который будет максимально содержать заданное в качестве аргумента количество элементов



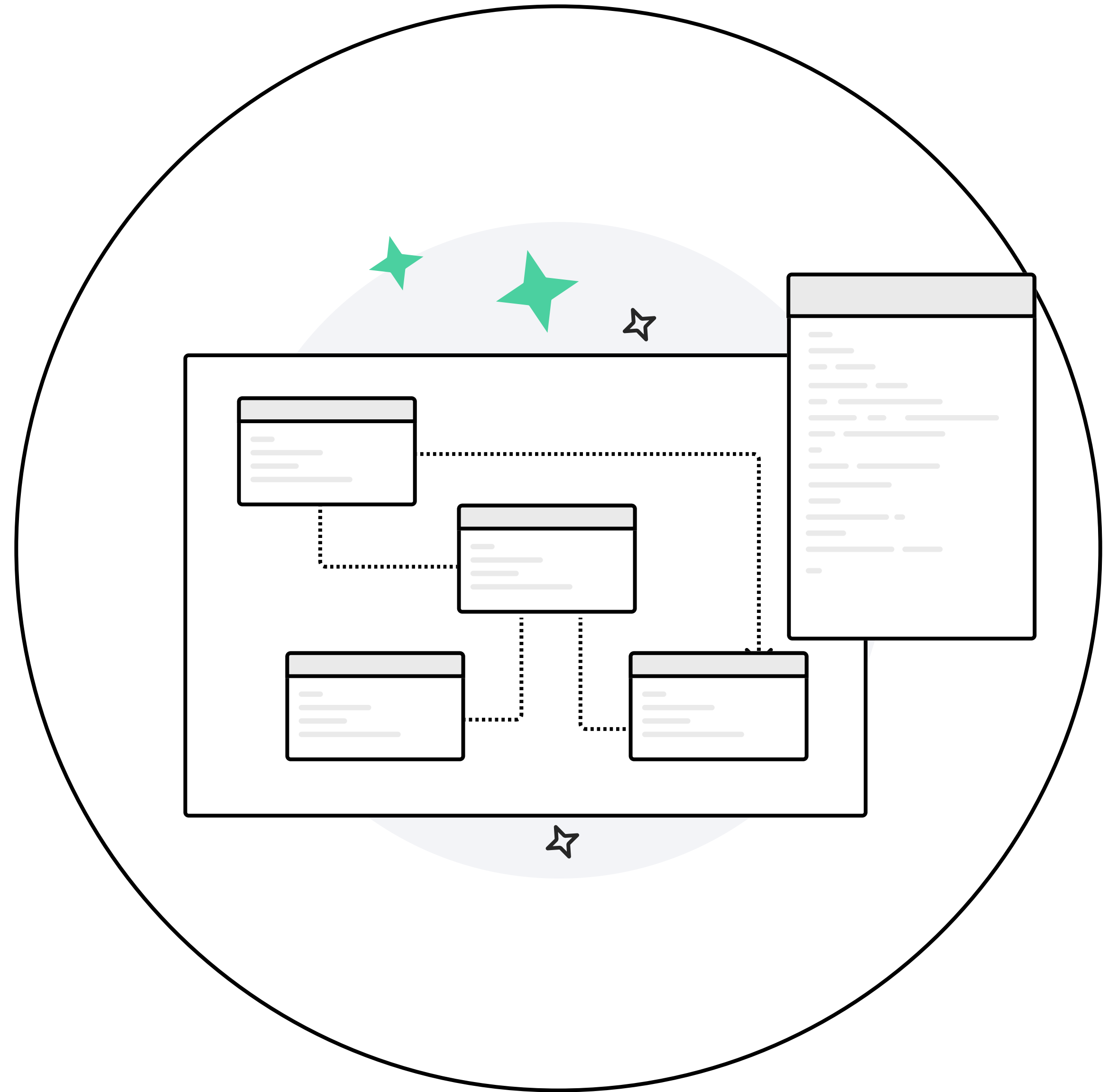
Limit

Пример: требуется вывести в консоль ограниченное число элементов листа

```
List<String> list = Arrays.asList("one", "two", "three", "one", "two");  
list = list.stream()  
    .limit(2)  
    .collect(Collectors.toList());  
System.out.println(list); // [one, two]
```

Sorted

Метод **sorted** применяется для сортировки элементов источника стрима



Sorted

В зависимости от типа используемого **компаратора** можно получить различный результат

```
List<String> list = Arrays.asList("9", "A", "Z", "1", "B", "Y", "4", "a", "c");
```

naturalOrder — сортировка элементов в естественном порядке

```
list.stream()  
  .sorted(Comparator.naturalOrder())  
  .forEach(System.out::print); // 149ABYZac
```

reverseOrder — сортировка элементов в обратном порядке

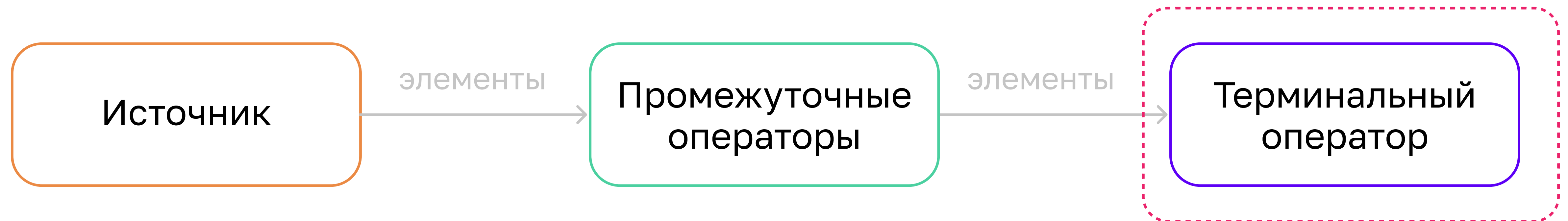
```
list.stream()  
  .sorted(Comparator.reverseOrder())  
  .forEach(System.out::print); // caZYBA941
```


Терминальные операции



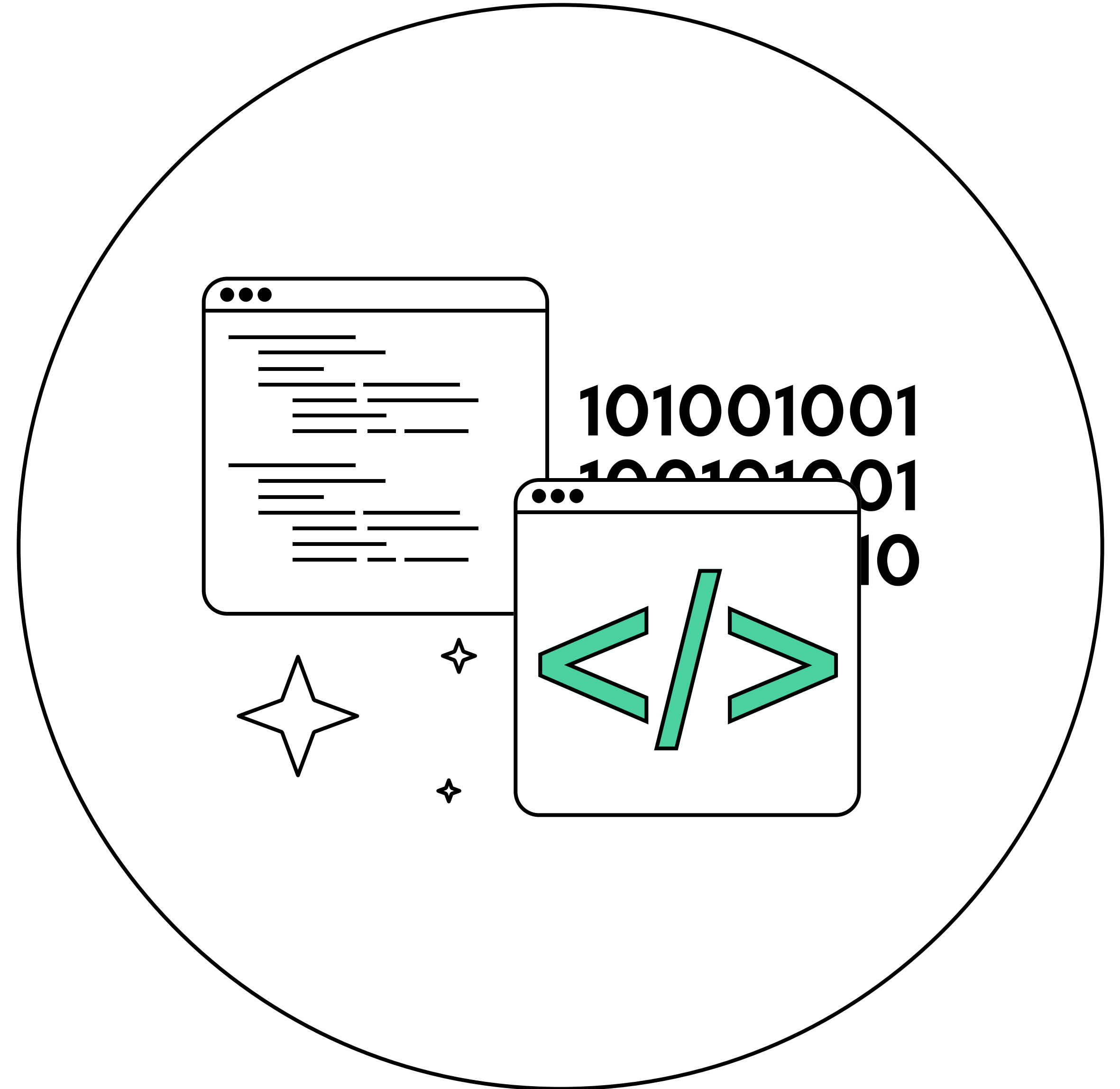
Терминальные операции

Вызов терминальной операцию в стриме завершает цепочку экземпляров Stream из промежуточных операций и возвращает результат



Match

Терминальные операции **match** применяются для проверки наличия совпадающего объекта в источнике стрима. В качестве аргумента используется предикат



Match

Match запускает внутреннюю итерацию стрима и применяет параметр предиката к каждому элементу:

- Метод **anyMatch** возвращает true, если предикат возвращает true для любого из элементов
- Метод **allMatch** возвращает true, если предикат возвращает true для всех элементов
- Метод **noneMatch** возвращает true, если предикат возвращает false для всех элементов

Match

Пример: выведем в консоль проверку элементов листа на наличие элемента, начинающего на «O»

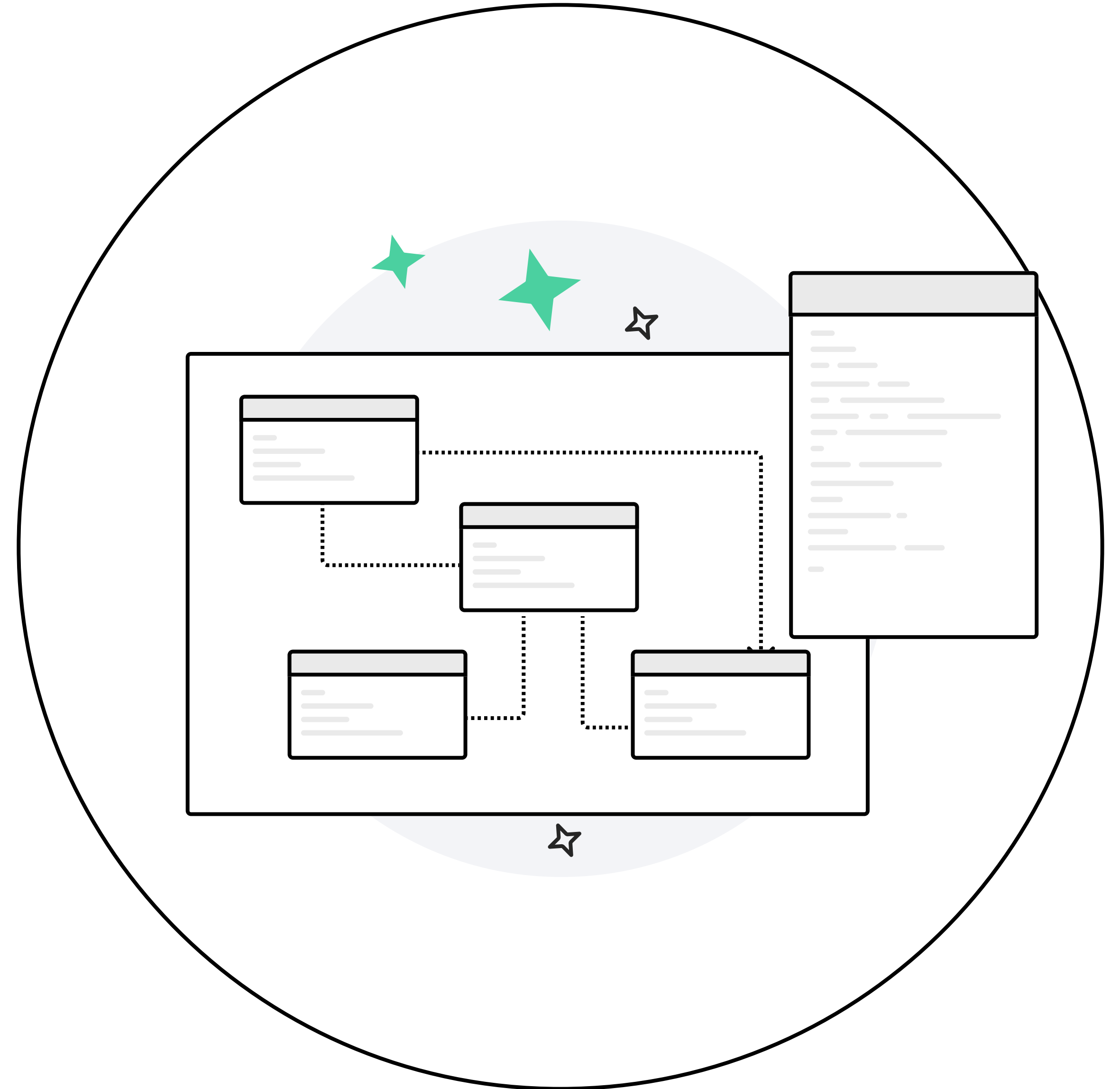
```
List<String> list = Arrays.asList("one", "two", "three");
```

```
boolean anyMatch = list.stream()  
    .anyMatch(value -> value.startsWith("o"));
```

```
System.out.println(anyMatch); // true
```


Collect

Метод **collect** — терминальная операция, которая запускает внутреннюю итерацию элементов и собирает элементы стрима в коллекцию



Collect

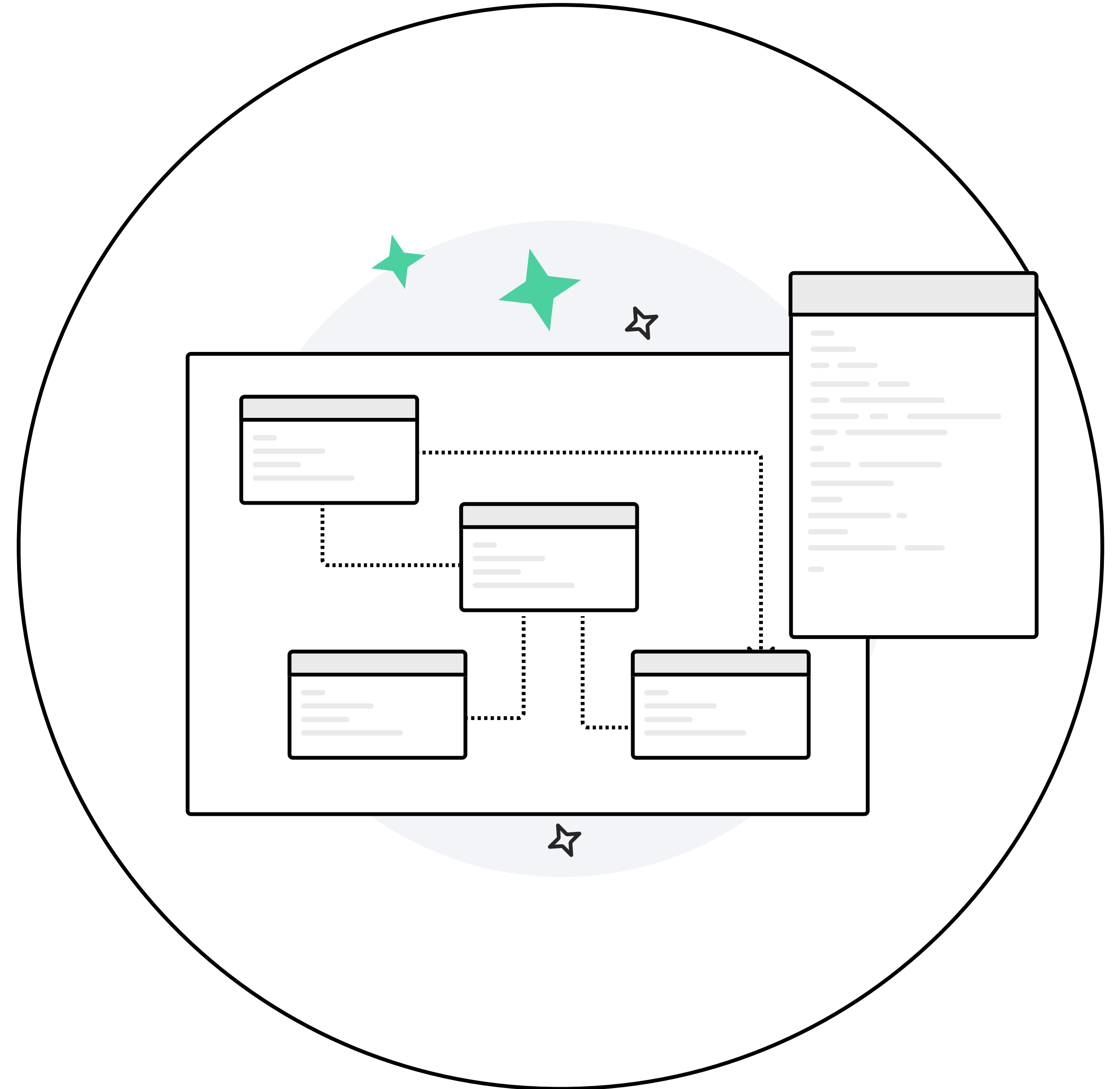
Пример: преобразуем элементы листа и занесём их в отдельный лист

```
List<String> list = Arrays.asList("one", "two", "three");
```

```
List<String> uppercaseList = list.stream()  
    .map(String::toUpperCase)  
    .collect(Collectors.toList());  
System.out.println(uppercaseList);
```

Count

Метод **count** — терминальная операция, которая запускает внутреннюю итерацию элементов и определяет количество элементов



Count

Пример: определим количество слов
в четверостишии

```
List<String> list = new ArrayList<>();  
list.add("И долго буду тем любезен я народу");  
list.add("Что чувства добрые я лирой пробуждал");  
list.add("Что в мой жестокий век восславил я Свободу");  
list.add("И милость к падшим призывал");  
  
long count = list.stream()  
    .flatMap(value -> Arrays.stream(value.split(" ")))  
    .count();  
System.out.println(count); // 26
```

Find

Метод **find** — терминальная операция, которая производит поиск элементов в стриме



Find

- Метод `findAny` может найти один элемент из стрима. Найденный элемент может быть из любой точки стрима
- Метод `findFirst` вернёт первый элемент, если таковой существует

Find

Пример: выведем в консоль первый элемент листа строчек. Проверим, существует ли результат

```
List<String> strings
    = Arrays.asList("Java Script", "Java 8", "Java 11", "Android", "Spring");

Optional<String> result = strings.parallelStream()
    .filter(s -> s.contains("Java"))
    .findFirst();

result.ifPresentOrElse(
    System.out::println,
    () -> System.out.println("There is no Java :(")
); // Java Script
```

Find

Пример: выведем в консоль первый элемент листа строчек. Проверим, существует ли результат

```
List<String> strings
    = Arrays.asList("Java Script", "Java 8", "Java 11", "Android", "Spring");

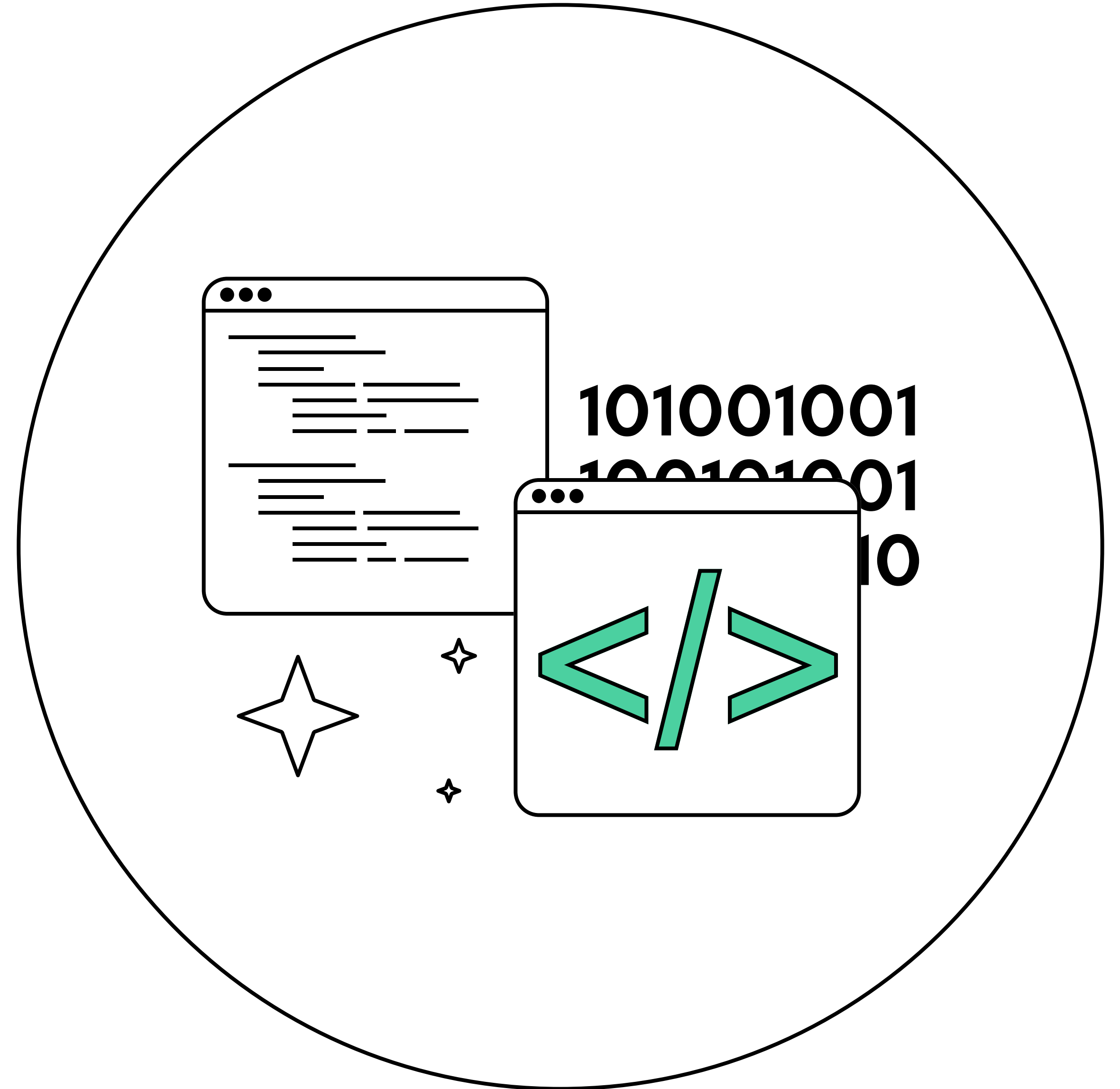
Optional<String> result = strings.parallelStream()
    .filter(s -> s.contains("Java"))
    .findFirst();

result.ifPresentOrElse(
    System.out::println,
    () -> System.out.println("There is no Java :(")
); // Java Script
```

Обратите внимание, что методы **find** возвращают тип **Optional**. Если стрим будет пустой, то метод не вернёт никакого результата. С помощью метода **ifPresent** проверим, был ли найден результат в **Optional**

ForEach

Метод **forEach** — терминальная операция, которая запускает внутреннюю итерацию элементов в стриме и применяет **Consumer** к каждому элементу в стриме



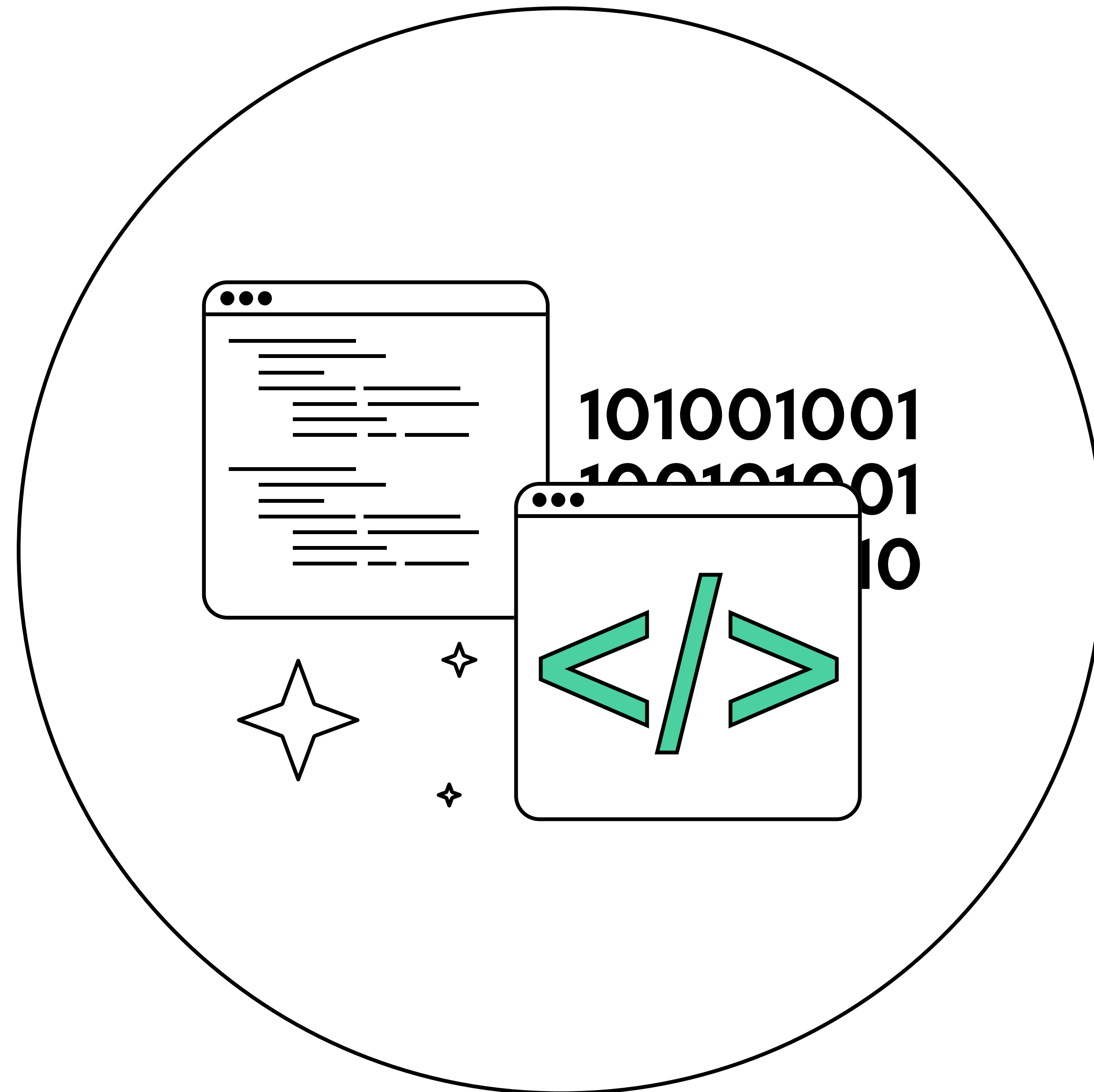
ForEach

Сам метод **forEach** возвращает **void**

```
Stream<String> stream = stringList.stream();  
stream.forEach(System.out::print);
```

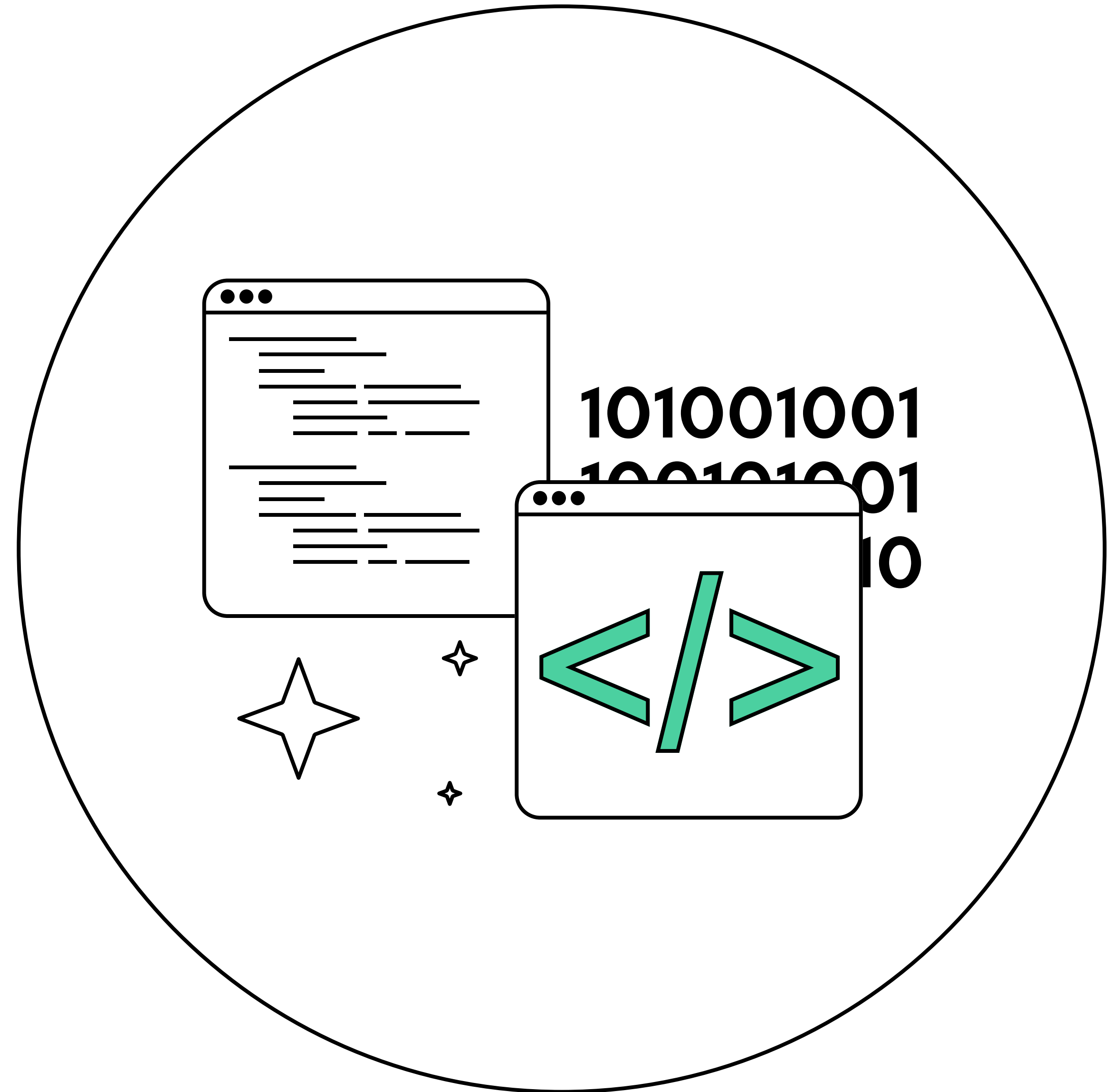
Min и max

Методы `min` и `max` — это терминальные операции, которые возвращают наименьший и наибольший элемент стрима



Min и max

Определение наименьшего и наибольшего элемента происходит с помощью передачи определённой имплементации компаратора в методы `min` и `max`



Min и max

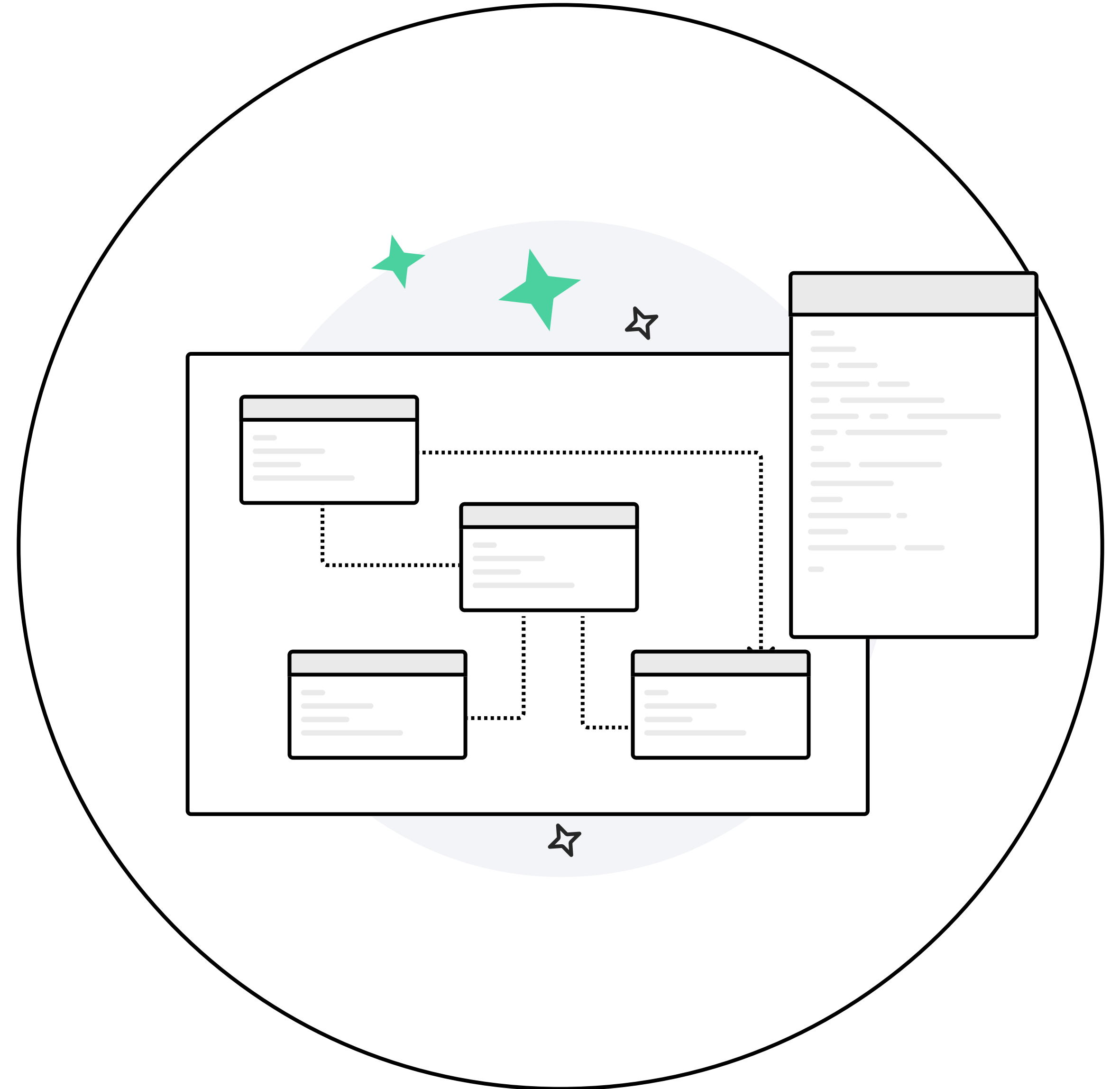
Пример: найдём наименьший элемент листа целых чисел

```
List<Integer> intList = Arrays.asList(1, 2, 5, 10);
Optional<Integer> min = intList.stream().min(Integer::compareTo);
if (min.isPresent()) {
    Integer minString = min.get();
    System.out.println(minString);
}
```

Обратите внимание, что метод `min` и `max` возвращает тип `Optional`, который может содержать или не содержать результат. Если стрим пустой, метод `get` пробросит исключение `NoSuchElementException`

ForEach

Метод **forEach** — терминальная операция, которая запускает внутреннюю итерацию элементов в стриме и применяет **Consumer** к каждому элементу в стриме



ForEach

Сам метод **forEach** возвращает **void**

```
Stream<String> stream = stringList.stream();  
stream.forEach(System.out::print);
```

Reduce

Метод **reduce** — это терминальная операция, которая может свести все элементы в стриме к одному элементу



Reduce

Reduce возвращает тип `Optional`. Этот необязательный параметр содержит значение, если оно есть, возвращаемое лямбда-выражением, переданным методу `reduce`. Получить значение можно с помощью `get`



Reduce

Пример: объединим элементы листа в одну строку через «+»

```
List<String> stringList = new ArrayList<String>();  
stringList.add("one");  
stringList.add("two");  
stringList.add("three");  
  
Optional<String> reduced = stringList.stream()  
    .reduce((value, combinedValue) -> combinedValue + " + " + value);  
reduced.ifPresent(System.out::println); // one + three + two
```

ToArray

Метод `toArray` — это терминальная операция, которая запускает внутреннюю итерацию элементов в стриме и возвращает массив `Object`, содержащий все элементы



ToArray

Пример: преобразуем лист строчек в массив объектов

```
List<String> stringList = new ArrayList<String>();  
stringList.add("One");  
stringList.add("Two");  
stringList.add("Thee");
```

```
Stream<String> stream = stringList.stream();  
Object[] objects = stream.toArray();
```